

Tópicos Especiais em Inteligência Computacional Aplicada a Segurança de Sistemas

Aula 04

Programa de Pós-graduação em Computação Aplicada
Núcleo de Desenvolvimento Amazônico em Engenharia
Prof. Dr. Caio C. Moreira

Ciclo de um sistema baseado em ML aplicado à segurança

- Coleta de dados
 - Logs, tráfego de rede, arquivos executáveis, eventos de execução.
 - Bases públicas de malwares (amostras brutas).
 - Bases Públicas com Características Extraídas (datasets prontos para ML)
- Pré-processamento
 - Limpeza e normalização dos dados.
 - Conversão de logs/arquivos em representações numéricas.
 - Tratamento de classes desbalanceadas (SMOTE, undersampling).
- **Extração de características (feature engineering)**
 - **Permissões solicitadas por aplicativos.**
 - **Chamadas de API/sistema.**
 - **Frequência de pacotes em tráfego de rede.**
 - **Strings e entropia em executáveis.**
- Treinamento do modelo
 - Escolha do algoritmo (supervisionado ou não supervisionado).
 - Ajuste de hiperparâmetros.
- Avaliação do modelo
 - Divisão em treino/teste (ou validação cruzada).
 - Métricas: Acurácia, Precisão, Recall, F1-score, AUC-ROC.
- Interpretação de resultados para validar a utilidade prática.

O que é Feature Engineering?

- Processo de transformar dados brutos em **atributos relevantes** para Machine Learning.
- Características bem escolhidas aumentam a precisão dos modelos.
- Em segurança, as features devem refletir comportamentos suspeitos ou padrões maliciosos.

Extração de características (feature engineering)

- Permissões solicitadas por aplicativos

- Aplicativos Android e outros sistemas móveis solicitam permissões (ex.: acesso a SMS, câmera, contatos).
- Permissões incomuns ou excessivas podem indicar comportamento malicioso.
- Feature: vetor binário (1 = permissão solicitada, 0 = não solicitada).
- Exemplo:
 - SEND_SMS=1, READ_CONTACTS=1, ACCESS_FINE_LOCATION=0.

Extração de características (feature engineering)

- Chamadas de API / Sistema

- Malware frequentemente faz sequências específicas de chamadas de API/sistema (ex.: criar processo, modificar registro, abrir socket).
- Essas sequências podem ser usadas como **assinaturas comportamentais**.
- Features comuns:
 - Contagem de chamadas específicas.
 - Sequência temporal (n-grams de chamadas).
 - Exemplo:
 - CreateFile → WriteFile → ExecuteProcess.
- Normalmente são extraídas através de uma sandbox
- Exemplo de monitoramento de Chamadas de Sistema (Linux)
 - `sudo strace -p `pidof nano` -f -e trace=write,read`

Extração de características (feature engineering)

- Frequência de pacotes em tráfego de rede
 - Tráfego malicioso costuma ter padrões diferentes do tráfego normal.
 - Possíveis Features extraídas:
 - Frequência de pacotes por protocolo (TCP, UDP, ICMP).
 - Tamanho médio e variância dos pacotes.
 - Taxa de envio (pacotes/s).
 - Útil em
 - Detecção de ataques DDoS (Distributed Denial of Service)
 - Análise de botnets e C&C (Command and Control)
 - Exfiltração de dados (Data Leakage)
 - Identificação de aplicações (Traffic Classification)
 - Detecção de port scanning

Extração de características (feature engineering)

- Strings e entropia em executáveis
 - Strings internas revelam indícios: URLs, nomes de funções, comandos.
 - Entropia mede o grau de aleatoriedade:
 - Baixa entropia → código comum.
 - Alta entropia → compressão ou obfuscação (muito usada em malware)
 - Features extraídas:
 - Contagem de strings legíveis.
 - Valor médio de entropia por seção do arquivo.
 - Exemplo:
 - Executável com alta entropia na seção .text pode indicar packers ou criptografia.
 - Exemplo de captura de strings no linux:
 - `strings nome_do_arquivo`

Atividade 04

1. Baixar 10 amostras de malware (virusshare.com) e 10 amostras de goodware
2. Criar um script em python para
 - a. Obter os caracteres imprimíveis (em palavras maiores que 3 caracteres) e humanamente legíveis de todo o conjunto.
 - b. Verificar a frequência de dessas palavras para filtrar as 100 mais frequentes.
 - c. Criar uma lista com as 100 palavras mais frequentes
3. Criar um script para, amostra por amostra, filtrar essas palavras e inserir no dataframe se a amostra contém ou não (0 ou 1) aquela palavra
 - a. Identificar cada amostra com uma coluna de nome, atribuir um número de identificação (ID) e um rótulo de identificação (malware ou goodware (1 ou 0))
4. Apresentar o código e o resultado na próxima aula